Diabetes Insight

Low Level Design Document

Application Type: Streamlit-Based AI Web Application

Core Modules: `app.py`, `predict.py`, `preprocessing.py`, `train_model.py`, `user_store.py`

Storage: CSV-based persistence using `users.csv`

Table of Contents

Section	Title
1	Introduction
2	Conventions
3	Design Constraints And Considerations
4	Software Architecture
5	Unit Processes
6	Global Description
7	Software Detailed Design For Batch Processes
8	Software Detailed Design For User Interface Processes
9	Alternate Design Methods Considered
10	Appendix A - Error Messages
11	Appendix B - Compliance LLD vs HLD
12	References

1. Introduction

1.1 Purpose

This Low Level Design document describes the implementation-level design of the **Diabetes Insight** system. It explains how the approved HLD modules are translated into concrete Python files, functions, data structures, and runtime interactions. The document supports development, academic evaluation, maintenance, and technical understanding of the application.

The LLD focuses on how the Streamlit user interface, CSV storage, model loading, prediction logic, training workflow, and runtime validation are organized in code. Each major module is mapped to responsibilities, interfaces, local data, global data usage, and error handling behavior.

1.2 Scope

The scope of this LLD includes detailed design of the user login flow, input form handling, prediction pipeline, model training batch process, CSV persistence, module interactions, exception management, and UI process logic. It applies to the current university project implementation in Python and Streamlit and assumes that the model is deployed as a pre-trained joblib file.

2. Conventions

The following conventions are used throughout this document:

1. Python files and functions are written in monospace, for example `app.py` and `load_model()`.
2. Client side refers to UI behavior in the browser rendered by Streamlit.
3. Server side refers to Python execution logic responsible for validation, inference, and persistence.
4. The term data store refers to the CSV persistence file `users.csv`.
5. Risk thresholds are interpreted as Low, Moderate, and High according to configured probability boundaries.
6. Session state refers to Streamlit runtime memory stored in `st.session_state`.

3. Design Constraints And Considerations

The implementation design must comply with the project constraints already approved in the SRS and HLD. The application is lightweight, file-based, and suitable for academic demonstration rather than production medical deployment. The key implementation concern is to keep the code simple, readable, and modular while ensuring correct inference and predictable UI behavior.

3.1 List Of Tasks On Server Side

Task Name	Description
Model loading	Load the pre-trained Random Forest model from disk using joblib and prepare it for runtime inference.
Prediction processing	Accept validated health input, structure it into model-compatible format, execute inference, and return label and probability.
CSV read/write	Create, read, update, and persist user and prediction data in `users.csv`.
Validation	Check required fields, numeric types, ranges, duplicate user conditions, and logical rules before prediction.
Risk classification	Convert the probability score into Low, Moderate, or High risk based on configured thresholds.
Session-backed control flow	Coordinate navigation between login, dashboard, and result presentation using runtime state.
Error handling	Detect missing files, invalid input, prediction failures, and storage issues and expose safe messages to the UI.

3.2 List Of Tasks On Client Side

Task Name	Description
UI rendering	Render login screen, dashboard widgets, buttons, status text, result panel, and logout controls.
Input handling	Collect name, gender, and medical attributes from Streamlit form components.
Button click events	Handle login, prediction, logout, and optional reset interactions.
Display results	Show prediction label, probability, risk class, and recommendation text in a user-readable format.

Gender-sensitive field control	Disable pregnancies input for male users and enable it for female users.
Feedback display	Show validation errors, duplicate-user warnings, missing-model errors, and save-status messages.

3.3 Event List

Event	Condition	Task Triggered
Login click	User enters name and gender and presses login	Login validation, duplicate check, session initialization
Predict click	User is logged in and has submitted the medical form	Input validation, preprocessing, inference, risk classification, CSV update
Gender selection	User selects male or female during login or active session	Pregnancies field state update
Logout	User presses logout button	Clear session state and route back to login page
Form update	User edits any health field	Update current input state and retain latest values for submission
Model missing	Prediction requested without available model file	Display system error and block inference

4. Software Architecture

4.1 Physical Task Vs HLD Unit Process Mapping

Physical Module / Task	HLD Unit Process	Detailed Responsibility
`app.py`	User Interaction Module	Render login and dashboard screens, maintain Streamlit flow, trigger prediction, and display results.
`predict.py`	Prediction Engine Module	Load model, validate inference input, execute prediction, and return probability and class.
`preprocessing.py`	Data Preparation Support	Define preprocessing pipeline used during model training and feature consistency alignment.
`train_model.py`	Batch Training Module	Train model candidates, evaluate performance, and save the best artifact.
`user_store.py`	Persistence Module	Create storage, read existing data, create users, and update prediction values in CSV.
`users.csv`	Data Store	Persist user details, health inputs, and prediction outputs.
`st.session_state`	Session State	Keep login status, current user, gender, active result, and UI state within the running app session.

4.2 Database Description

The application uses CSV instead of a relational database. The file `users.csv` acts as the operational data store and contains login identity data, health features, prediction values, and result metadata. The schema is fixed and must remain consistent across reads and writes.

Column	Description
`name`	User identifier used during login.
`gender`	Selected gender used for form behavior.
`pregnancies`	Pregnancy count or zero for male users.
`glucose`	Glucose value used for prediction.
`blood_pressure`	Blood pressure value.

`skin_thickness`	Skin thickness value.
`insulin`	Insulin value.
`bmi`	Body Mass Index.
`diabetes_pedigree_function`	Family-history-linked risk factor.
`age`	Age in years.
`prediction`	Predicted class label.
`probability`	Predicted diabetic-class probability.
`risk_class`	Low, Moderate, or High risk.
`timestamp`	Time at which the prediction record was stored.

4.3 External Interface Design

Streamlit UI Interface: accepts login and medical inputs, triggers actions, and presents results.

Model Interface: provides a callable prediction entry point that accepts one structured record and returns class and probability.

CSV Interface: provides functions to create the file, read rows, detect duplicates, and insert or update data safely.

5. Unit Processes

5.1 Process Inputs

Input Parameter	Source	Type	Validation Rule
Name	Login form	Text	Required and non-empty
Gender	Login form	Enum / Text	Required, male or female
Pregnancies	Prediction form	Integer	Editable only for female users; forced to zero for male users
Glucose	Prediction form	Numeric	Required and within accepted range
Blood Pressure	Prediction form	Numeric	Required and within accepted range
Skin Thickness	Prediction form	Numeric	Required and within accepted range
Insulin	Prediction form	Numeric	Required and within accepted range
BMI	Prediction form	Numeric	Required and within accepted range
Diabetes Pedigree Function	Prediction form	Numeric	Required and valid
Age	Prediction form	Integer	Required and positive

5.2 Process Outputs

Output	Description
Prediction label	Binary model output that indicates diabetic or non-diabetic classification.
Probability	Confidence score for the diabetic class returned from the model or derived from probability support.

Risk classification	Low, Moderate, or High based on configured threshold boundaries.
Interpretation text	User-friendly explanation of the risk result for display in the UI.
CSV record	Stored row containing identity, medical values, prediction, probability, and timestamp.

5.3 Data Store

The principal data store is `users.csv`. It is used to persist user information and the most recent or cumulative prediction details, depending on the storage policy implemented in `user_store.py`. The data store is intentionally simple and optimized for academic demonstration rather than transactional production use.

5.4 Processing Specification

1. User opens the application and logs in with name and gender.
2. Application stores login details in `st.session_state`.
3. Dashboard is rendered with health input controls.
4. Pregnancies field is enabled for female users and disabled for male users.
5. User enters all medical values and presses the predict button.
6. Input validation is performed for completeness, data type, and range correctness.
7. Validated values are converted into a one-record dictionary or DataFrame.
8. The prediction module loads the model using joblib if not already cached.
9. The feature vector is ordered according to the training schema.
10. The model generates class prediction and diabetic-class probability.
11. Probability is converted into Low, Moderate, or High risk classification.
12. UI displays the label, probability, and interpretation text.
13. CSV storage module saves or updates the user prediction record.

5.5 Exception, Signal And Error Handling

Exception / Condition	**Cause**	**Handling Strategy**
Missing model	Model file absent or unreadable	Display error and stop inference immediately
Invalid input	Missing, non-numeric, or out-of-range field values	Reject submission and show corrective feedback

Duplicate user	Existing name reused when uniqueness is enforced	Show warning and block new-user creation
Prediction failure	Inference exception or incompatible input schema	Show generic prediction failure message and log internally if available
CSV creation failure	Missing path or permission issue	Warn the user and maintain UI stability
CSV write failure	Permission, lock, or schema inconsistency issue	Display storage warning and preserve current session
Session inconsistency	Dashboard accessed without valid login state	Redirect back to login screen

6. Global Description

6.1 Function Description

Function	Description
`load_model()`	Loads the serialized Random Forest model from disk using joblib and returns the in-memory model.
`predict_single()`	Accepts one validated record, performs inference, and returns label, probability, and risk class.
`create_user()`	Creates a new logical user entry in the CSV data store.
`upsert_user_prediction()`	Updates existing user data or inserts prediction values with timestamp into storage.
`classify_risk()`	Maps the diabetic probability to Low, Moderate, or High risk.
`ensure_user_store()`	Creates `users.csv` with the expected headers if it does not already exist.

6.2 Interface

Caller Module	Called Module / Function	Purpose
`app.py`	`create_user()`	Create a user after successful login if a new record is needed.
`app.py`	`predict_single()`	Request model inference after form submission.
`app.py`	`upsert_user_prediction()`	Persist prediction values to CSV after result generation.
`predict.py`	`load_model()`	Load model artifact before prediction begins.
`train_model.py`	`preprocessing.py`	Build preprocessing steps for training pipeline construction.

6.3 Global Data Structure Usage

Global Data Structure	Purpose	Used In

`st.session_state`	Maintain login state, gender, active user, current screen, and latest result.	`app.py`
Input dictionary	Temporary structure holding submitted medical values before conversion to model input.	`app.py`, `predict.py`
CSV schema definition	Defines the column order expected in `users.csv`.	`user_store.py`
Feature column order	Ensures training-time and inference-time alignment of attributes.	`predict.py`, `train_model.py`

6.4 Processing Logic

The full application flow is linear and state-controlled. The application starts with login, validates and stores the session, renders the dashboard, collects health values, validates them, loads the trained model, predicts the probability and class, classifies the risk, displays the result, and saves the output to CSV. Repeat prediction is supported within the same user session, while logout clears the state and returns the user to the login page.

6.5 Exception / Error Handling

Error handling is designed to be defensive and user-safe. Invalid input is blocked before prediction, missing model artifacts stop inference immediately, duplicate users can be prevented where required, and storage errors never result in raw internal tracebacks being shown to the user. In all cases, the system attempts to preserve the session and keep the UI responsive.

7. Software Detailed Design For Batch Processes

7.1 Task Description

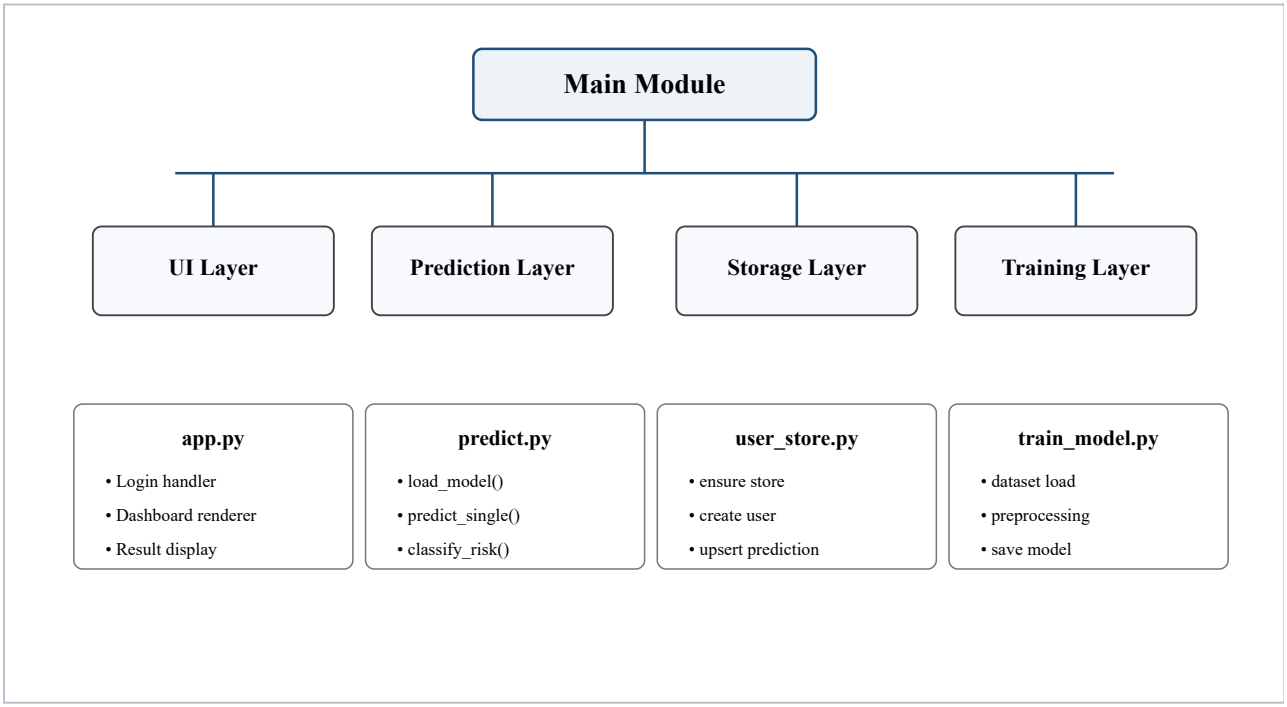
The main batch task in the system is model training. It is executed offline by the project developer rather than the end user. The task reads the diabetes dataset, applies preprocessing, trains one or more candidate classifiers, evaluates them, and saves the best-performing model artifact for runtime use.

7.2 Task Type

Task	Type	Description
Model training	Batch	Offline task executed to produce the deployable model artifact.
Preprocessing fit	Batch	Training-time transformation fitting using the source dataset.
Model saving	Batch	Persist final trained model to joblib file.
Prediction request	Real-time	User-triggered inference for a single record.
CSV update	Real-time	Runtime save of user prediction data.

7.3 Structure Chart (TEXT)

Figure 1. Structure Chart



7.4 Database Usage

Data Store	Access Type	Used By	Purpose
`data/diabetes.csv`	Read	`train_model.py`	Training dataset source
`models/best_model.joblib`	Write	`train_model.py`	Persist the trained best model
`models/best_model.joblib`	Read	`predict.py`	Runtime model loading for inference
`data/users.csv`	Read / Write	`user_store.py`, `app.py`	Store user and prediction data

7.5 Library Function / Common Function Used

Library	Purpose
`pandas`	Dataset loading, DataFrame operations, CSV persistence
`scikit-learn`	Random Forest classifier, training pipeline, evaluation support
`joblib`	Model serialization and loading
`streamlit`	UI rendering and session handling
`datetime`	Timestamp generation for saved records

7.6 Global Data Structure Description

Global data structures include the session dictionary maintained by Streamlit, the ordered list of feature columns expected by the model, the CSV column schema used by the persistence layer, and the model object loaded from the joblib file during inference.

7.7 Call Convention

Modules communicate through standard Python function calls. Input arguments are passed as primitive values, dictionaries, or DataFrames, while returned values are passed as dictionaries, Booleans, status objects, or model instances. Naming and return conventions are kept simple to make the code easy to inspect during academic review.

7.8 Module Specifications

7.8.1 app.py

Description: Main Streamlit controller responsible for page rendering, user flow, event handling, and integration of prediction and storage services.

Interface: Uses Streamlit widgets for input and calls `create_user()`, `predict_single()`, and `upsert_user_prediction()`.

File Usage: Indirectly accesses `users.csv` and the model file via dependent modules.

Global Data Usage: Reads and writes `st.session_state` for login status, user identity, gender, and latest result.

Local Data: Temporary form values, event flags, user input dictionary, result objects.

Logic: Initialize state, display login, validate identity, render dashboard, capture health input, invoke prediction, display result, support logout.

Error Handling: Displays login warnings, validation errors, duplicate-user warnings, model errors, and storage-save failures.

7.8.2 predict.py

Description: Inference module that encapsulates model loading, feature preparation, prediction execution, and risk classification logic.

Interface: Exposes `load_model()`, `predict_single()`, and `classify_risk()`.

File Usage: Reads `models/best_model.joblib`.

Global Data Usage: Uses configured model path, feature ordering, and threshold constants.

Local Data: One-record DataFrame, predicted label, probability variable, risk class string.

Logic: Verify model exists, load model, structure input, call prediction methods, compute probability, map risk, return result.

Error Handling: Handles missing model, malformed input, incompatible schema, and inference-time exceptions.

7.8.3 preprocessing.py

Description: Defines the training-time data transformation rules and provides preprocessing objects to the training module.

Interface: Exposes preprocessing builder functions and feature configuration helpers.

File Usage: Typically interacts with training data indirectly through `train_model.py`.

Global Data Usage: Feature lists, numeric or categorical field grouping, preprocessing configuration.

Local Data: Transformers, imputer configuration, scalers, and encoder definitions.

Logic: Identify columns, define numeric transformations, define categorical transformations if required, and build a reusable transformer pipeline.

Error Handling: Raises or reports missing column definitions and transformer construction issues.

7.8.4 train_model.py

Description: Offline batch-training module that reads the source dataset, trains candidate models, compares them, and saves the best model.

Interface: Exposes a training entry point and internal functions for model evaluation and serialization.

File Usage: Reads `data/diabetes.csv` and writes `models/best_model.joblib`.

Global Data Usage: Dataset path, target column, feature columns, output model path.

Local Data: Train/test splits, model candidates, evaluation scores, fitted best model.

Logic: Load dataset, split features and target, apply preprocessing, train candidate models, evaluate, choose best, save artifact.

Error Handling: Handles missing dataset, missing target column, training-time errors, and joblib serialization failures.

7.8.5 user_store.py

Description: CSV persistence layer that ensures file existence, manages schema, creates users, detects duplicates, and saves prediction results.

Interface: Exposes `ensure_user_store()`, `create_user()`, `get_user()`, and `upsert_user_prediction()`.

File Usage: Reads and writes `data/users.csv`.

Global Data Usage: CSV path and ordered schema columns.

Local Data: DataFrame rows, duplicate masks, update masks, row dictionaries.

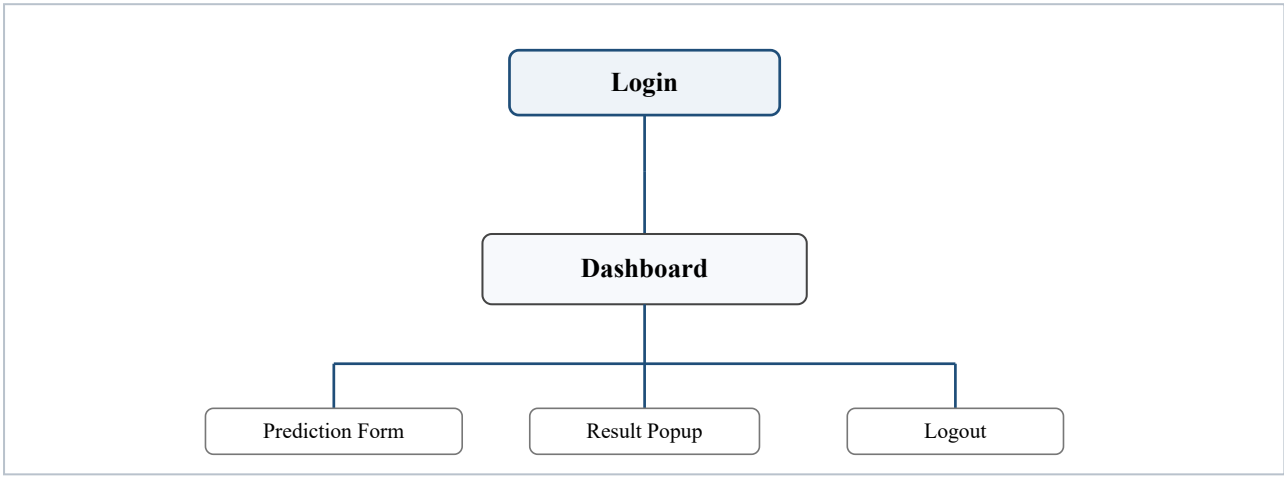
Logic: Ensure file exists, load rows, check existence, create or update target row, write back to CSV.

Error Handling: Handles file absence, malformed data, duplicate users, and write-permission failures.

8. Software Detailed Design For User Interface Processes

8.1 Menu Hierarchy

Figure 2. Menu Hierarchy



8.2 Class Hierarchy Diagram (TEXT)

```
AppController
|
+-- SessionManager
|
+-- UIFormManager
|
+-- PredictionService
|   |
|   +-- RiskClassifier
|
+-- UserRepository
```

8.3 Class Description

Logical Class / Module	Description
AppController	Controls page routing, event handling, and coordination of UI with backend modules.
SessionManager	Encapsulates reading and writing of Streamlit session state entries.

PredictionService	Groups model loading, validation, prediction, and risk classification behavior.
UserRepository	Represents CSV persistence behavior such as user creation and prediction updates.

8.4 Task Description

8.4.1 Screen Layout

Login page: project title, name input, gender selector, login button, warning area.

Main dashboard: grouped health inputs, dynamic pregnancies field, predict button, logout control, and optional reset action.

Result popup / result section: prediction label, probability, risk class, interpretation text, and continuation option.

8.4.2 Screen Item Description

Screen Item	Type	Purpose	Validation / Behavior
Name	Text input	Capture user identity	Required
Gender	Select / radio	Control profile and pregnancy logic	Required
Pregnancies	Number input	Accept pregnancy count	Enabled only for female users
Glucose	Number input	Medical predictor input	Required and ranged
BMI	Number input	Medical predictor input	Required and ranged
Age	Number input	Medical predictor input	Required and positive
Predict	Button	Trigger inference	Allowed only when form is valid
Logout	Button	End current session	Clears state and returns to login

8.4.3 Functions Invoked

The UI invokes `create_user()`, `ensure_user_store()`, `predict_single()`, `classify_risk()`, and `upsert_user_prediction()` depending on the user action.

8.4.4 Function Description

Login-related functions initialize the session and optionally create the user in storage. Prediction-related functions validate input, perform inference, classify risk, and update CSV records. UI helper functions render forms, messages, and state-aware navigation.

8.4.5 Interface

The UI passes a single assembled input structure to the prediction module and receives a structured result object in return. The storage module receives the current user name, gender, health values, and result values when persistence is requested.

8.4.6 Global Data Usage

The UI depends on `st.session_state` for login status, user identity, gender, temporary form values, and last prediction result. This allows Streamlit reruns to preserve context across interactions.

8.4.7 Processing Logic

The UI flow starts at the login page, transitions to the dashboard after valid session setup, accepts health input, performs prediction on button click, displays the result, stores the output, and supports another prediction or logout without restarting the application.

8.4.8 Error Handling

UI-level error handling consists of required-field warnings, duplicate-user messages, range-validation feedback, model unavailability alerts, prediction failure notifications, and storage failure warnings.

8.4.9 Database Usage (table)

UI Action	CSV Usage
Login	Read user store to validate uniqueness or existing profile behavior
First user creation	Insert initial record in `users.csv`
Prediction completion	Update or append prediction values with timestamp
Repeat prediction	Persist latest values for the active user

8.4.10 Libraries Used

Library	Usage
`streamlit`	Widgets, layout, state, and event handling
`pandas`	Packaging input into DataFrame form and supporting CSV work
`joblib`	Indirect model loading through the prediction service

`scikit-learn`	Indirect runtime use of the trained model object

9. Alternate Design Methods Considered

Why Streamlit over React: Streamlit was selected because it allows the project to stay fully in Python, which is a strong fit for a machine-learning-centered academic system. A React frontend would require a separate API layer, additional state management, and more deployment complexity. Streamlit provides a faster, clearer, and more maintainable path for university evaluation.

Why CSV over a database: CSV storage satisfies the lightweight data persistence requirement, is easy to inspect during project review, avoids database configuration, and remains sufficient for the expected low data volume. A relational database would improve scalability but would add complexity beyond the needs of the current scope.

Why Random Forest over other algorithms: Random Forest is well suited to tabular medical data, supports probability output, handles nonlinear relationships effectively, and is simple to integrate with scikit-learn pipelines. It offers a practical balance between performance and implementation simplicity for a student AI project.

10. Appendix A - Error Messages

Error Code	Message
E001	Name is required.
E002	Gender selection is required.
E003	User already exists.
E004	Invalid input value detected.
E005	One or more required medical fields are missing.
E006	Input value is outside the allowed range.
E007	Model file not found.
E008	Failed to load prediction model.
E009	Prediction failed. Please try again.
E010	Unable to save prediction history.
E011	User store file is missing or corrupted.
E012	Session expired or invalid. Please login again.
E013	Pregnancies field is not applicable for male users.
E014	Training dataset not found.
E015	Failed to save trained model.

11. Appendix B - Compliance LLD vs HLD

HLD Area	LLD Section	Compliance Description
System environment	Sections 3 and 4	HLD components are refined into server-side and client-side tasks and interfaces.
Behavioral description	Sections 3.3, 5.4, 6.4, 8.4.7	HLD behavior is translated into event-driven runtime flow.
Process model	Sections 4, 5, 7	High-level modules are decomposed into files, tasks, functions, and storage use.

Data model	Sections 4.2, 5.3, 6.3, 7.4	Logical data design is implemented as fixed CSV schema and update logic.
Object-oriented design	Sections 6.2, 7.8, 8.2, 8.3	Logical components are represented as cooperating modules and service-like classes.
Prototype and UI flow	Section 8	Detailed screen flow and control behavior are defined for implementation.
Construction strategy	Sections 7.5 and 9	Library usage and design decisions align with the selected construction method.
Reliability requirements	Sections 5.5, 6.5, 7.8, 8.4.8	Error handling and state management support reliability objectives.
Maintainability	Sections 6, 7, 8	Modular structure and clear interfaces support future updates and academic clarity.

> **Conclusion:** This LLD provides a complete implementation view of Diabetes Insight and is aligned with the approved HLD. It is suitable for university submission, code implementation, and technical review.

12. References

Sl. No	Reference	Purpose
1	IEEE recommended practices for software design and software documentation	Used as the structural basis for preparing the LLD in a formal academic format.
2	Project Software Requirements Specification (SRS) for Diabetes Insight	Used to derive detailed module behavior, constraints, interfaces, and validation needs.
3	Project High Level Design (HLD) for Diabetes Insight	Used to refine architectural components into implementation-level modules and processes.
4	Streamlit official documentation	Used for UI rendering concepts, session state handling, widgets, and event-driven application flow.
5	scikit-learn documentation	Used for Random Forest model behavior, preprocessing pipelines, training utilities, and inference conventions.
6	joblib documentation	Used for model serialization and loading of the trained machine learning artifact.
7	pandas documentation	Used for CSV reading, writing, and tabular data manipulation in the persistence layer.
8	Python standard library documentation	Used for file handling, path management, and timestamp generation support.
9	Project source modules: `app.py`, `predict.py`, `preprocessing.py`, `train_model.py`, `user_store.py`	Used as the implementation reference base for the detailed design descriptions.
10	Project data and artifact files: `data/diabetes.csv`, `data/users.csv`, `models/best_model.joblib`	Used to define the batch process, runtime inference, and data-store design.